Data Modelling in MongoDB

As we have seen from the Introduction section, the data in MongoDB has a flexible schema. Unlike in SQL databases, where you must have a table's schema declared before inserting data, MongoDB's collections do not enforce document structure. This sort of flexibility is what makes MongoDB so powerful.

When modeling data in Mongo, keep the following things in mind

1. What are the needs of the application – Look at the business needs of the application and see what data and the type of data needed for the application. Based on this, ensure that the structure of the document is decided accordingly.
2. What are data retrieval patterns – If you foresee a heavy query usage then consider the use of indexes in your data model to improve the efficiency of queries.
3. Are frequent inserts, updates and removals happening in the database? Reconsider the use of indexes or incorporate sharding if required in your data modeling design to improve the efficiency of your overall MongoDB environment.

Difference between MongoDB & RDBMS

Below are some of the key term differences between MongoDB and RDBMS

RDBMS	MongoDB	Difference
Table	Collection	In RDBMS, the table contains the columns and rows which are used to store the data whereas, in MongoDB, this same structure is known as a collection. The collection contains documents which in turn contains Fields, which in turn are key-value pairs.

Row	Document	In RDBMS, the row represents a single, implicitly structured data item in a table. In MongoDB, the data is stored in documents.
Column	Field	In RDBMS, the column denotes a set of data values. These in MongoDB are known as Fields.
Joins	Embedded documents	In RDBMS, data is sometimes spread across various tables and in order to show a complete view of all data, a join is sometimes formed across tables to get the data. In MongoDB, the data is normally stored in a single collection, but separated by using Embedded documents. So there is no concept of joins in MongoDB.

Apart from the terms differences, a few other differences are shown below

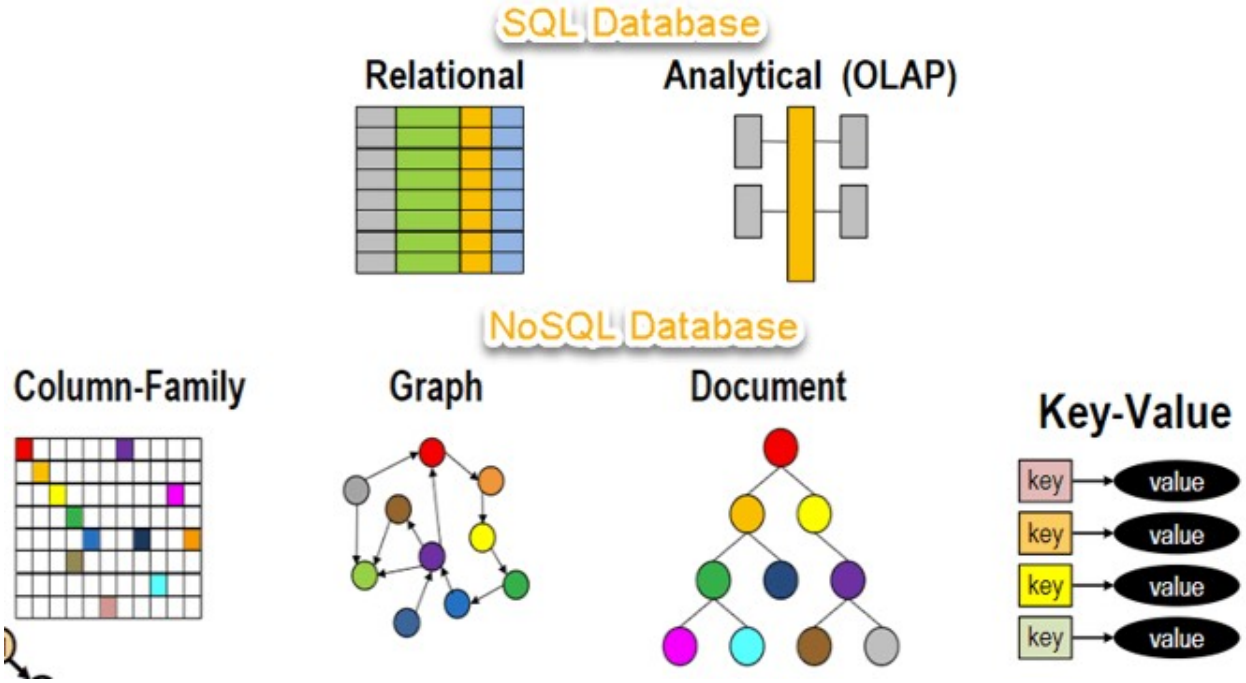
1. Relational databases are known for enforcing data integrity. This is not an explicit requirement in MongoDB.
2. RDBMS requires that data be normalized first so that it can prevent orphan records and duplicates Normalizing data then has the requirement of more tables, which will then result in more table joins, thus requiring more keys and indexes.As databases start to grow, performance can start becoming an issue. Again this is not an explicit requirement in MongoDB. MongoDB is flexible and does not need the data to be normalized first.

What is NoSQL?

NoSQL Database is a non-relational Data Management System, that does not require a fixed schema. It avoids joins, and is easy to scale. The major purpose of using a NoSQL database is for distributed data stores with humongous data storage needs. NoSQL is used for Big data and real-time web apps. For example, companies like Twitter, Facebook and Google collect terabytes of user data every single day.

NoSQL database stands for “Not Only SQL” or “Not SQL.” Though a better term would be “NoREL”, NoSQL caught on. Carl Strozz introduced the NoSQL concept in 1998.

Traditional RDBMS uses SQL syntax to store and retrieve data for further insights. Instead, a NoSQL database system encompasses a wide range of database technologies that can store structured, semi-structured, unstructured and polymorphic data. Let’s understand about NoSQL with a diagram in this NoSQL database tutorial:

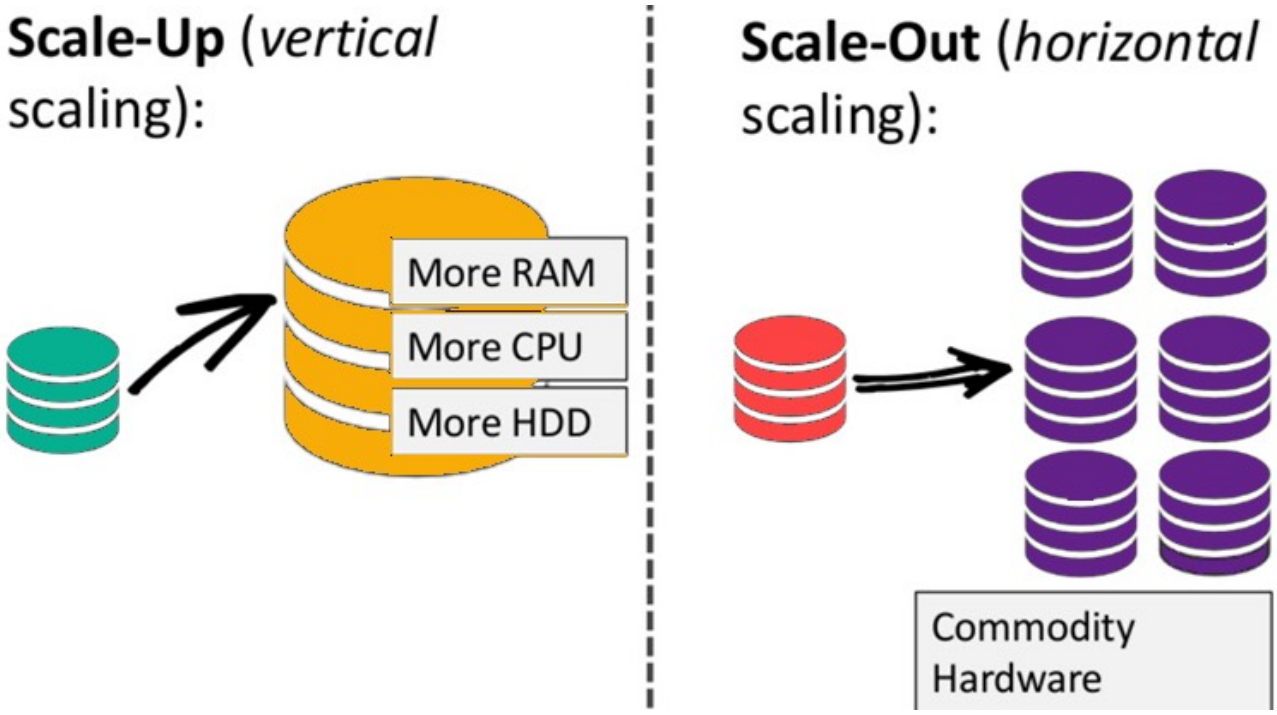


Why NoSQL?

The concept of NoSQL databases became popular with Internet giants like Google, Facebook, Amazon, etc. who deal with huge volumes of data. The system response time becomes slow when you use RDBMS for massive volumes of data.

To resolve this problem, we could “scale up” our systems by upgrading our existing hardware. This process is expensive.

The alternative for this issue is to distribute database load on multiple hosts whenever the load increases. This method is known as “scaling out.”



NoSQL database is non-relational, so it scales out better than relational databases as they are designed with web applications in mind.

Features of NoSQL

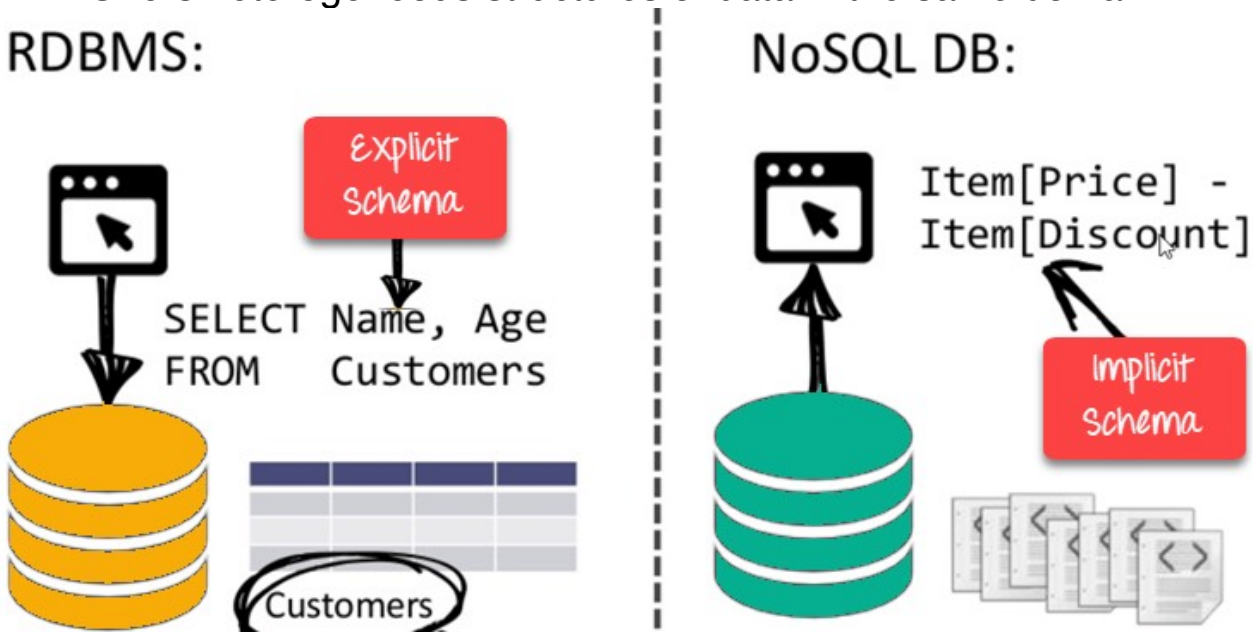
Non-relational

- NoSQL databases never follow the relational model
- Never provide tables with flat fixed-column records
- Work with self-contained aggregates or BLOBs

- Doesn’t require object-relational mapping and data normalization
- No complex features like query languages, query planners,referential integrity joins, ACID

Schema-free

- NoSQL databases are either schema-free or have relaxed schemas
- Do not require any sort of definition of the schema of the data
- Offers heterogeneous structures of data in the same domain



NoSQL is Schema-Free

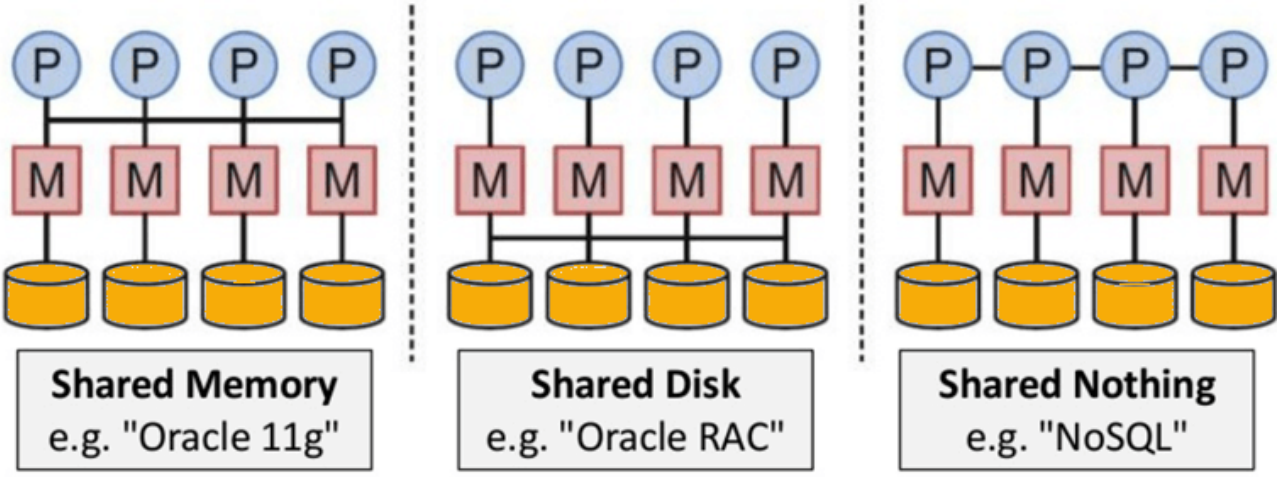
Simple API

- Offers easy to use interfaces for storage and querying data provided
- APIs allow low-level data manipulation & selection methods
- Text-based protocols mostly used with HTTP REST with JSON
- Mostly used no standard based NoSQL query language
- Web-enabled databases running as internet-facing services

Distributed

- Multiple NoSQL databases can be executed in a distributed fashion
- Offers auto-scaling and fail-over capabilities
- Often ACID concept can be sacrificed for scalability and throughput

- Mostly no synchronous replication between distributed nodes Asynchronous Multi-Master Replication, peer-to-peer, HDFS Replication
- Only providing eventual consistency
- Shared Nothing Architecture. This enables less coordination and higher distribution.



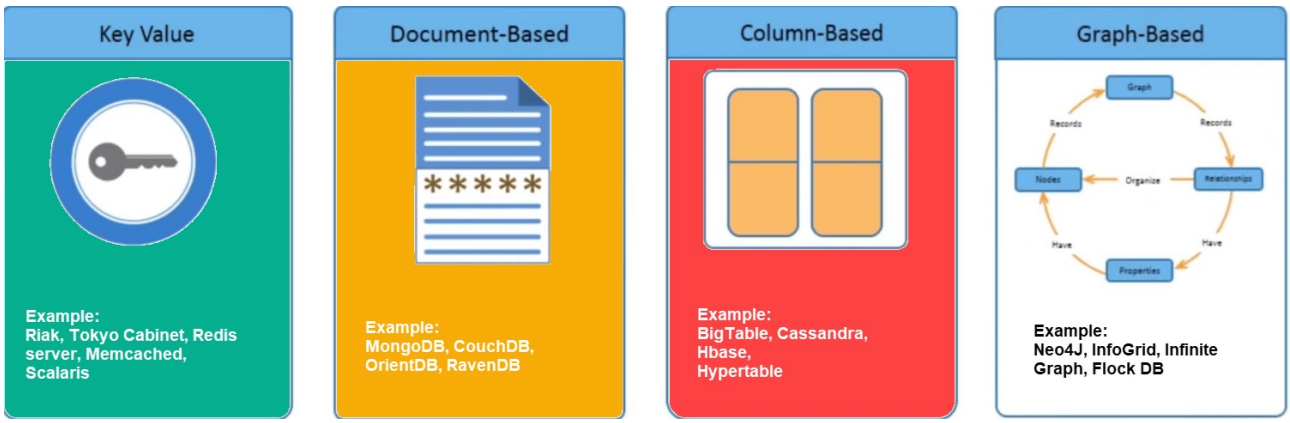
NoSQL is Shared Nothing.

Types of NoSQL Databases

NoSQL Databases are mainly categorized into four types: Key-value pair, Column-oriented, Graph-based and Document-oriented. Every category has its unique attributes and limitations. None of the above-specified database is better to solve all the problems. Users should select the database based on their product needs.

Types of NoSQL Databases:

- Key-value Pair Based
- Column-oriented Graph
- Graphs based
- Document-oriented



Key Value Pair Based

Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.

Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB(Binary Large Objects), string, etc.

For example, a key-value pair may contain a key like “Website” associated with a value like “Google”.

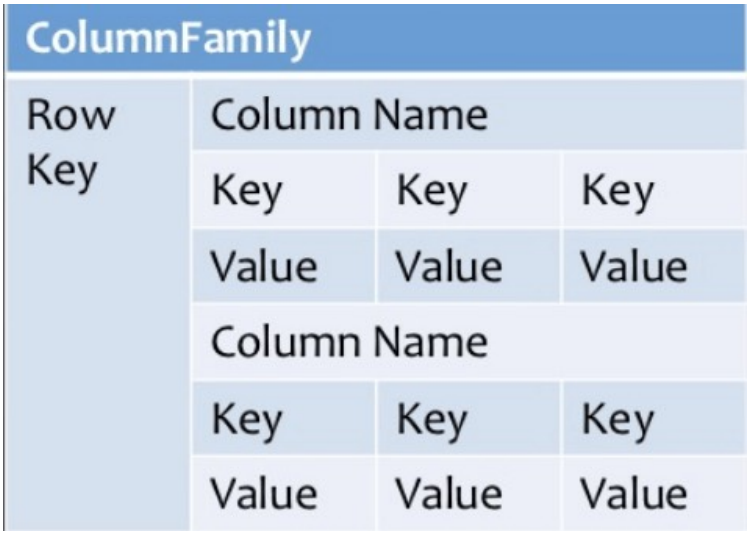
Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

It is one of the most basic NoSQL database example. This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc. Key value stores help the developer to store schema-less data. They work best for shopping cart contents.

Redis, Dynamo, Riak are some NoSQL examples of key-value store DataBases. They are all based on Amazon’s Dynamo paper.

Column-based

Column-oriented databases work on columns and are based on BigTable paper by Google. Every column is treated separately. Values of single column databases are stored contiguously.



Column based NoSQL database

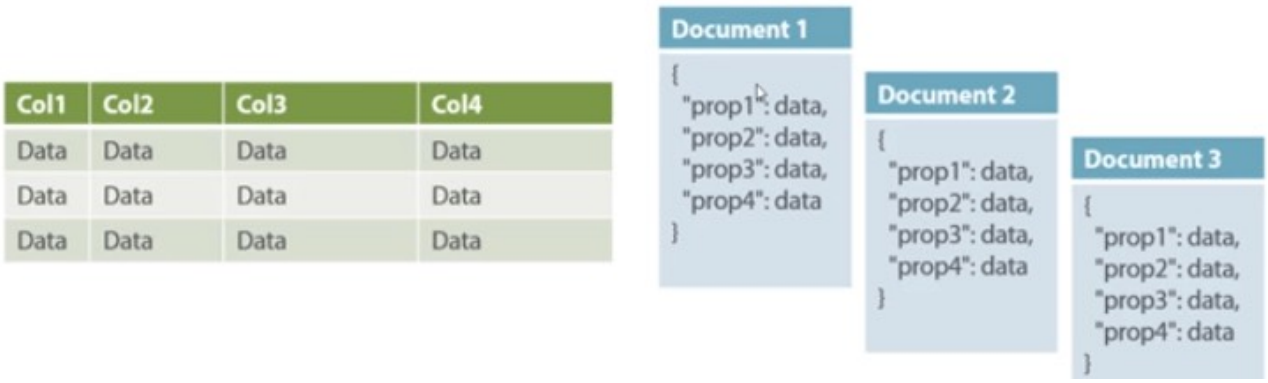
They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.

Column-based NoSQL databases are widely used to manage data warehouses, business intelligence, CRM, Library card catalogs,

HBase, Cassandra, HBase, Hypertable are NoSQL query examples of column based database.

Document-Oriented:

Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document. The document is stored in JSON or XML formats. The value is understood by the DB and can be queried.



Relational Vs. Document

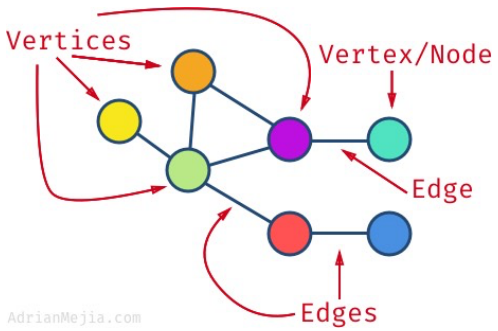
In this diagram on your left you can see we have rows and columns, and in the right, we have a document database which has a similar structure to JSON. Now for the relational database, you have to know what columns you have and so on. However, for a document database, you have data store like JSON object. You do not require to define which make it flexible.

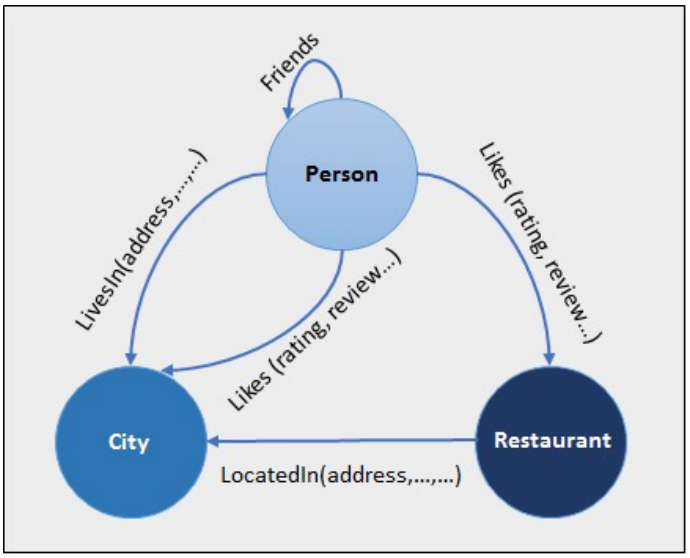
The document type is mostly used for CMS systems, blogging platforms, real-time analytics & e-commerce applications. It should not use for complex transactions which require multiple operations or queries against varying aggregate structures.

Amazon SimpleDB, CouchDB, MongoDB, Riak, Lotus Notes, MongoDB, are popular Document originated DBMS systems.

Graph-Based

A graph type database stores entities as well the relations amongst those entities. The entity is stored as a node with the relationship as edges. An edge gives a relationship between nodes. Every node and edge has a unique identifier.





Compared to a relational database where tables are loosely connected, a Graph database is a multi-relational in nature. Traversing relationship is fast as they are already captured into the DB, and there is no need to calculate them.

Graph base database mostly used for social networks, logistics, spatial data.

Neo4J, Infinite Graph, OrientDB, FlockDB are some popular graph-based databases.

Query Mechanism tools for NoSQL

The most common data retrieval mechanism is the REST-based retrieval of a value based on its key/ID with GET resource

REST - Representational state transfer is a software architectural style that describes a uniform interface between physically separate components, often across the Internet in a client-server architecture

Document store Database offers more difficult queries as they understand the value in a key-value pair. For example, CouchDB allows defining views with MapReduce

What is the CAP Theorem?

CAP theorem is also called brewer's theorem. It states that is impossible for a distributed data store to offer more than two out of three guarantees

1. Consistency
2. Availability
3. Partition Tolerance

Consistency:

The data should remain consistent even after the execution of an operation. This means once data is written, any future read request should contain that data. For example, after updating the order status, all the clients should be able to see the same data.

Availability:

The database should always be available and responsive. It should not have any downtime.

Partition Tolerance:

Partition Tolerance means that the system should continue to function even if the communication among the servers is not stable. For example, the servers can be partitioned into multiple groups which may not communicate with each other. Here, if part of the database is unavailable, other parts are always unaffected.

Eventual Consistency

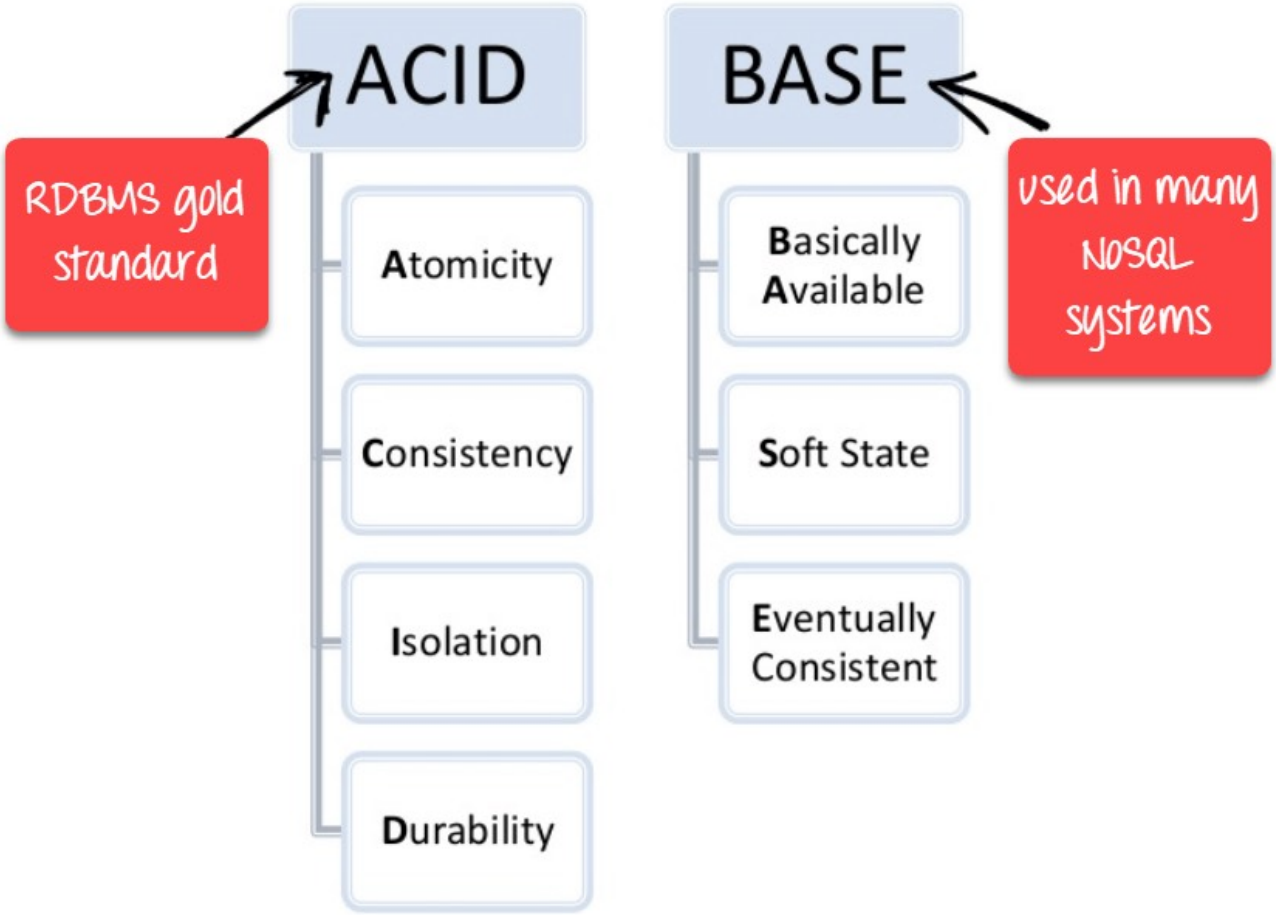
The term "eventual consistency" means to have copies of data on multiple machines to get high availability and scalability. Thus, changes made to any data item on one machine has to be propagated to other replicas.

Data replication may not be instantaneous as some copies will be updated immediately while others in due course of time. These copies

may be mutually, but in due course of time, they become consistent. Hence, the name eventual consistency.

BASE: Basically Available, Soft state, Eventual consistency

- Basically, available means DB is available all the time as per CAP theorem
- Soft state means even without an input; the system state may change
- Eventual consistency means that the system will become consistent over time



Advantages of NoSQL

- Can be used as Primary or Analytic Data Source
- Big Data Capability
- No Single Point of Failure
- Easy Replication
- No Need for Separate Caching Layer
- It provides fast performance and horizontal scalability.

- Can handle structured, semi-structured, and unstructured data with equal effect
- Object-oriented programming which is easy to use and flexible
- NoSQL databases don't need a dedicated high-performance server
- Support Key Developer Languages and Platforms
- Simple to implement than using RDBMS
- It can serve as the primary data source for online applications.
- Handles big data which manages data velocity, variety, volume, and complexity
- Excels at distributed database and multi-data center operations
- Eliminates the need for a specific caching layer to store data
- Offers a flexible schema design which can easily be altered without downtime or service disruption

Disadvantages of NoSQL

- No standardization rules
- Limited query capabilities
- RDBMS databases and tools are comparatively mature
- It does not offer any traditional database capabilities, like consistency when multiple transactions are performed simultaneously.
- When the volume of data increases it is difficult to maintain unique values as keys become difficult
- Doesn't work as well with relational data
- The learning curve is stiff for new developers
- Open source options so not so popular for enterprises.

The use Command

MongoDB **use DATABASE_NAME** is used to create database. The command will create a new database if it doesn't exist, otherwise it will return the existing database.

Syntax
Basic syntax of **use DATABASE** statement is as follows –

```
use DATABASE_NAME
```

Example
If you want to use a database with name **<mydb>**, then **use DATABASE** statement would be as follows –

```
>use mydb
```

```
switched to db mydb
```
To check your currently selected database, use the command **db**

```
>db
mydb
```
If you want to check your databases list, use the command **show dbs**.

```
>show dbs
local     0.78125GB
test      0.23012GB
```
Your created database (mydb) is not present in list. To display database, you need to insert at least one document into it.

```
>db.movie.insert({"name":"tutorials point"})
>show dbs
local      0.78125GB
mydb       0.23012GB
test       0.23012GB
```
In MongoDB default database is test. If you didn't create any database, then collections will be stored in test database.

MongoDB Create Database

Use Database method:

There is no create database command in MongoDB. Actually, MongoDB do not provide any command to create database.

It may be look like a weird concept, if you are from traditional SQL background where you need to create a database, table and insert values in the table manually.

Here, in MongoDB you don't need to create a database manually because MongoDB will create it automatically when you save the value into the defined collection at first time.

How and when to create database

If there is no existing database, the following command is used to create a new database.

Syntax:

1. use DATABASE_NAME

If the database already exists, it will return the existing database.

Let' take an example to demonstrate how a database is created in MongoDB

. In the following example, we are going to create a database "sample".

See this example

1. >use sample

```
Swithched to db sample
```

To **check the currently selected database**, use the command db:

1. >db

```
sample
```

MongoDB Drop Database

The dropDatabase command is used to drop a database. It also deletes the associated data files. It operates on the current database.

Syntax:

1. db.dropDatabase()

This syntax will delete the selected database. In the case you have not selected any database, it will delete default "test" database.

MongoDB Drop Database

The dropDatabase command is used to drop a database. It also deletes the associated data files. It operates on the current database.

Syntax:

1. db.dropDatabase()

This syntax will delete the selected database. In the case you have not selected any database, it will delete default "test" database.

```
{ "dropped": "sample ", "ok": 1}
```

Now check the list of databases:

1. >show dbs

```
local 0.078GB
```

The createCollection() Method

MongoDB **db.createCollection(name, options)** is used to create collection.

Syntax

Basic syntax of **createCollection()** command is as follows –

```
db.createCollection(name, options)
```

In the command, **name** is name of collection to be created. **Options** is a document and is used to specify configuration of collection.

Parameter	Type	Description
Name	String	Name of the collection to be created
Options	Document	(Optional) Specify options about memory size and indexing

Options parameter is optional, so you need to specify only the name of the collection. Following is the list of options you can use –

Field	Type	Description
capped	Boolean	(Optional) If true, enables a capped collection. Capped collection is a fixed size collection that automatically overwrites its oldest entries when it reaches its maximum size. **If you specify true, you need to specify size parameter also.**

autoIndexId	Boolean	(Optional) If true, automatically create index on _id field.s Default value is false.
size	number	(Optional) Specifies a maximum size in bytes for a capped collection. **If capped is true, then you need to specify this field also.**
max	number	(Optional) Specifies the maximum number of documents allowed in the capped collection.

While inserting the document, MongoDB first checks size field of capped collection, then it checks max field.

Examples
Basic syntax of **createCollection()** method without options is as follows –

```
>use test
switched to db test
>db.createCollection("mycollection")
{ "ok" : 1 }
>
```

You can check the created collection by using the command **show collections**.

```
>show collections
mycollection
system.indexes
```

The following example shows the syntax of **createCollection()** method with few important options –

```
> db.createCollection("mycol", { capped : true,
autoIndexId : true, size : 6142800, max : 10000 } )
{
"ok" : 0,
"errmsg" : "BSON field 'create.autoIndexID' is an
unknown field.",
"code" : 40415,
"codeName" : "Location40415"
}
>
```

What Is a Capped Collection in MongoDB?

Fixed-size collections are called capped collections in MongoDB. While creating a collection, the user must specify the collection's maximum size in bytes and the maximum number of documents that it would store. If more documents are added than the specified capacity, the existing ones are overwritten.

Capped collection in MongoDB supports high-throughput operations used for insertion and retrieval of documents based on the order of insertion. Capped collection working is similar to circular buffers, i.e., there is a fixed space allocated for the capped collection. The oldest documents are overwritten to make space for the new documents in the collection once the fixed size gets exhausted.

Creating Capped Collection in MongoDB

Use the createCollection command along with the capped option to create a capped collection. Specify the collection's maximum size in bytes as follows:

```
>db.createCollection("cappedLogCollection",
{capped:true,size:10000})
```

To limit the number of documents that can be included in the capped collection, use the max parameter as shown below:

```
>db.createCollection("cappedLogCollection",
{capped:true,size:10000,max:1000})
```

How to Check Whether a Collection is Capped or Not?

You can check whether a collection is capped using the isCapped() method. This method returns true as the output if the specified collection is a capped one. Otherwise, it returns false.

To check whether a given collection is capped or not, use the isCapped function as follows:

```
>db.cappedLogCollection.isCapped()
```

How to Convert a Collection to a Capped?

If there is a normal collection, you can change it to capped by using the convertToCapped command. To convert an existing collection to capped, use the following code:

```
>db.runCommand({"convertToCapped":"posts",size:10000})
```

Advantages of Capped Collections in MongoDB

- It returns documents in insertion order without the need for an index and provides greater insertion throughput.
- Capped collections enable changes that match the original document size, ensuring the position of the document does not change on the disk.
- Holding log files.

Disadvantages of Capped Collections

- It cannot be shared.
- An update operation fails when the document exceeds the original size of the capped collection in MongoDB.
- It is not possible to delete documents from a capped collection. You can delete all records using the following command: { emptycapped: Collection_name }

In MongoDB, you don't need to create collection. MongoDB creates collection automatically, when you insert some document.

```
>db.mycollection.insert({"name" : "mycollection"}),
WriteResult({ "nInserted" : 1 })
>show collections
mycol
mycollection
system.indexes
Sample
>
```

The drop() Method

MongoDB's **db.collection.drop()** is used to drop a collection from the database.

Syntax
Basic syntax of **drop()** command is as follows –

```
db.COLLECTION_NAME.drop()
```

Example
First, check the available collections into your database **mydb**.

```
>use mydb
switched to db mydb
>show collections
mycol
```

```
mycollection
system.indexes
Sample
>
```

Now drop the collection with the name **mycollection**.

```
>db.mycollection.drop()
true
>
```

Again check the list of collections into database.

```
>show collections
mycol
system.indexes
Sample
>
```

drop() method will return true, if the selected collection is dropped successfully, otherwise it will return false.

Introduction to the mongo shell

The mongo shell is an interactive JavaScript interface to MongoDB. The mongo shell allows you to manage data in MongoDB as well as carry out administrative tasks.

MongoDB client

The mongo shell is a MongoDB client. By default, the mongo shell connects to the test database on the MongoDB server and assigns the database connection to the global variable called db.

The db variable allows you to see the current database:

```
> db
test
```

Note that when you type a variable or an expression on the command line, the mongo shell will evaluate it and returns the result.

Besides supporting JavaScript syntax, the mongosh shell provides you with useful commands that easily interact with the MongoDB database server.

For example, you can use the shows dbs command to list all the databases on the server:

```
test> show dbs
admin       41 kB
config    73.7 kB
local     81.9 kB
Code language: CSS (css)
```

The output shows three databases on the server.

To switch the current database to another, you use the use <database> command. For example, the following command switches the current database to the bookdb database:

```
test> use bookdb
switched to db bookdb
Code language: PHP (php)
```

Note that you can switch to a database that does not exist. In this case, MongoDB will automatically create that database when you first save the data in it.

The mongo shell now assigns bookdb to the db variable:

```
> db
bookdb
```

Now, you can access the books collection from the bookstore database via the db variable like this:

```
> db.books
bookstore.books
Code language: CSS (css)
```

MongoDB - Insert Single Document in a Collection using insertOne()

In MongoDB, a collection represents a table in RDBMS and a document is like a record in a table. Learn how to insert a single document into a collection.

MongoDB provides the following methods to insert documents into a collection:

1. insertOne() - Inserts a single document into a collection.
2. insert() - Inserts one or more documents into a collection.
3. insertMany() - Insert multiple documents into a collection.

insertOne()

Use the `db.<collection>.insertOne()` method to insert a single document in a collection. `db` points to the current database, `<collection>` is existing or a new collection name.

Syntax:

```
db.collection.insertOne(document, [writeConcern])
```

Parameters:

1. document: A document to insert into the collection.
2. writeConcern: Optional. A document expressing the write concern to override the default write concern.

The following inserts a document into `employees` collection.

Example: insertOne() Copy

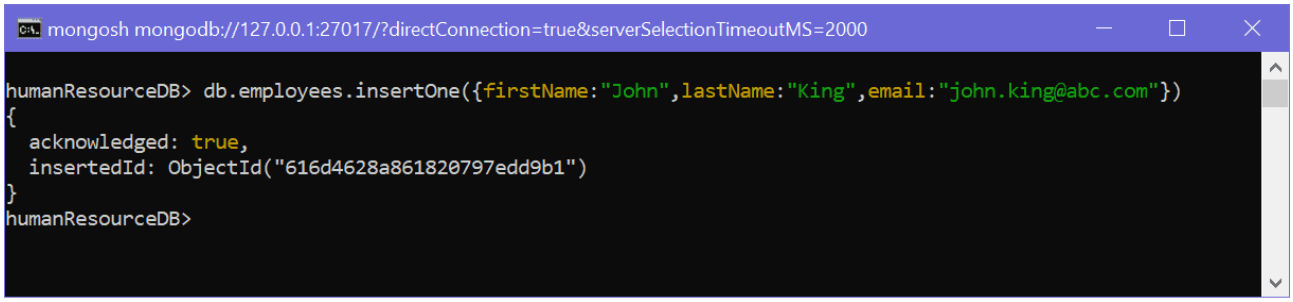
```
db.employees.insertOne({
    firstName: "John",
    lastName: "King",
    email: "john.king@abc.com"
})
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("616d44bea861820797edd9b0")
}
```

In the above example, we passed a document to the `insertOne()` method. Notice that we haven't specified `_id` field. So, MongoDB inserts a document to a collection with the auto-generated unique `_id` field. It returns an object with a boolean field `acknowledged` that indicates whether the insert operation is successful or not, and `insertedId` field with the newly inserted `_id` value.

The following shows the insert operation in the mongosh shell:



InsertOne() in mongosh Shell